

Chatbot para búsqueda de vehículos usados

Usando Retrieval Augmented Generation (RAG)

Proyecto Final Deep Learning • Febrero 2026
Nicolas Fripp - Diego Aguiar - Bruno Moraes

El Problema: Limitaciones de la Búsqueda Tradicional

1

Brecha Semántica

La búsqueda por palabras clave no entiende la intención del usuario. Ejemplo: "auto para familia grande", quizás no te muestra camionetas porque no escribiste la palabra exacta "camioneta"

2

Consultas Complejas

Necesidades multi-atributo difíciles de expresar con filtros. Es un lío marcar 20 casillas como en algunos lugares de autos

3

Sin Contexto

Cada búsqueda es independiente - sin memoria conversacional

4

Descubrimiento Limitado

Se pierden vehículos relevantes sin palabras exactas

Nuestra Solución: Arquitectura RAG

Combina búsqueda semántica con IA conversacional

1

Embeddings

Transforma texto en vectores que capturan significado semántico

text-embedding-3-small

2

Búsqueda Vectorial

Encuentra vehículos similares muy rápido usando FAISS

3

Generación

respuestas en lenguaje natural

gpt-4o-mini

El Dataset

38,531

Publicaciones de Vehículos

Algunas Características por Vehículo

- fabricante, modelo, año, precio
- motor (combustible, cilindrada, tipo), transmisión
- tipo de carrocería, color, tracción
- kilometraje, garantía, estado, etc

Ejemplo de Conversión a Texto

"Subaru Outback del año 2010. Precio: \$9,200 USD. Motor: gasoline de 2.5 litros. Transmisión: automatic. Kilometraje: 190,000 km. Tipo de carrocería: universal. Color: silver. Tracción: all. Estado: owned."

Cómo Funciona: El Pipeline

1

Convertir a Texto

Cada vehículo -> descripción en lenguaje natural

2

Crear Embeddings

OpenAI embeddings -> vectores de 1.536 dimensiones

3

Construir Vector Store

FAISS indexa los 38.531 vehículos

4

Consulta del Usuario

"SUVs económicos" -> se embebe y se busca

5

Recuperar + Generar

Top 5 resultados -> gpt-4o-mini sintetiza la respuesta

Rápido y económico: un solo embedding + una sola llamada LLM por consulta

Cómo armamos la llamada al LLM?

"Sos un asistente experto en autos que ayuda a los usuarios a encontrar información sobre vehículos.

Usa el siguiente contexto para responder la pregunta del usuario. El contexto contiene información sobre autos reales del dataset.

Contexto:

`{context}`

Pregunta: `{question}`

Instrucciones:

- Respondé en español de forma clara y concisa
- Si encontrás varios autos relevantes, mencioná los más destacados (máximo 5)
- Incluí detalles específicos como marca, modelo, año, precio cuando sea relevante
- Si no hay información suficiente en el contexto, decí que no encontraste autos que coincidan
- Podés hacer comparaciones si el usuario lo pide

Respuesta: "

Ventajas Clave de RAG

1

Súper Rápido

Búsquedas en segundos

2

Económico

1 embedding + 1 llamada LLM por consulta

3

Búsqueda Semántica

encuentra resultados relevantes sin palabras exactas

4

Persistente

Vector store guardado en disco - recarga instantánea, sin re-embedding

5

Simple

Menos trabajo frente a un fine tuning por ejemplo

6

Conversacional

Mantiene historial de conversación para seguimientos con contexto

Detalles de Implementación

Stack Tecnológico

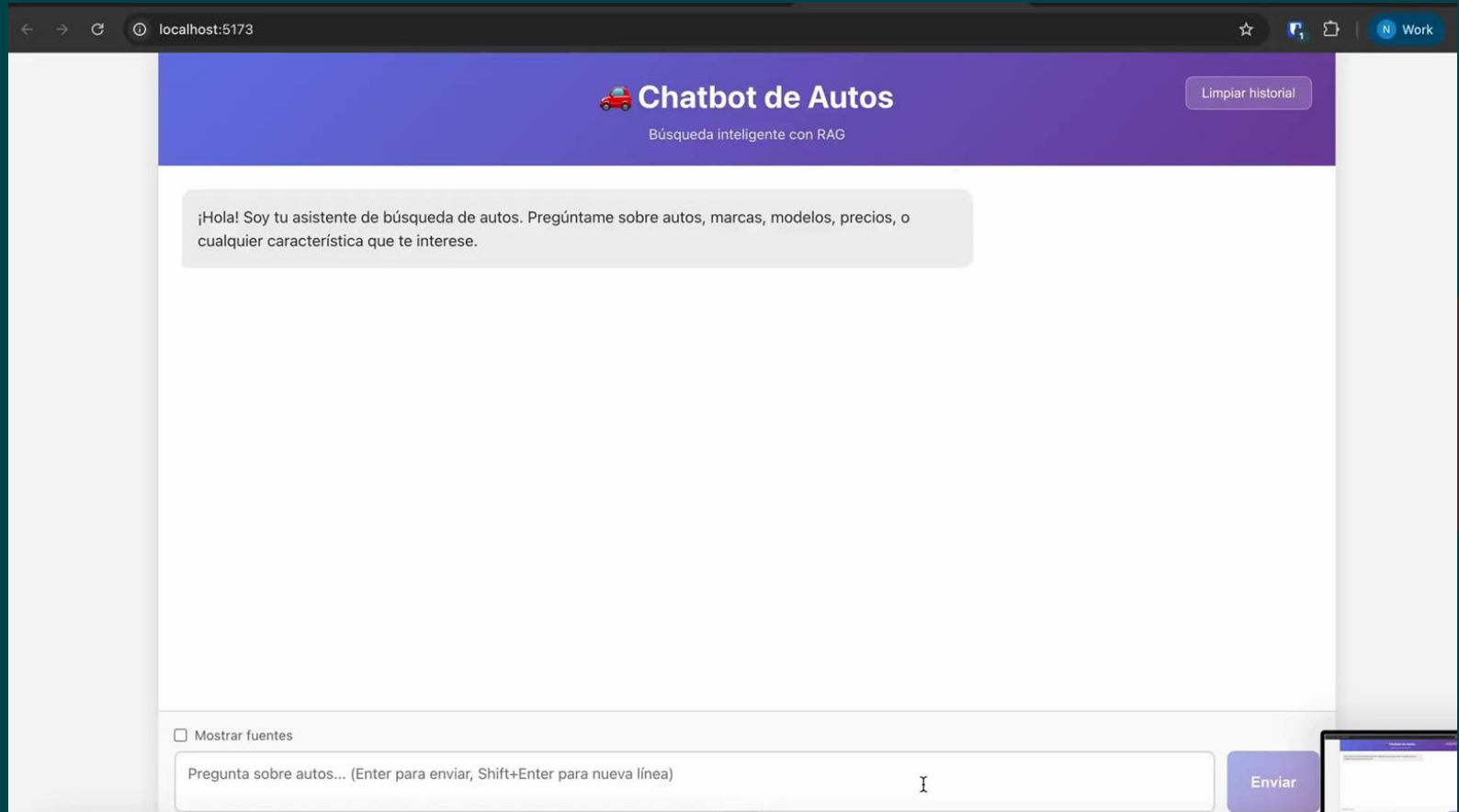
- LangChain: Framework de orquestación
- OpenAI Embeddings: text-embedding-3-small
- FAISS: Vector store optimizado para CPU
- gpt-4o-mini: Generación de respuestas
- Pandas: Procesamiento de datos
- Python + Google Colab: Desarrollo

Aplicación Real:

Chatbot para Automotoras

- Implementamos un sistema completo con backend en Python (FastAPI) y frontend en React
- Chatbot conversacional que permite a las automotoras ofrecer búsqueda inteligente a sus clientes

Demo



Conclusiones

1

RAG es ideal para tareas de recuperación de datos

2

Los embeddings pre-entrenados capturan relaciones semánticas sin necesidad de fine-tuning

3

FAISS permite búsqueda lista para producción con infraestructura mínima

4

La memoria conversacional agrega valor para refinar búsquedas iterativas

Gracias

¿Preguntas?